



Universitat
de les Illes Balears

TRABAJO DE FIN DE GRADO

DISEÑO E IMPLEMENTACIÓN DE UNA PLATAFORMA WEB PARA LA VENTA DE PRODUCTOS DIGITALES CON GESTIÓN DE LICENCIAS Y PAGOS INTEGRADOS

Joaquin Esperon Solari

Grau d'Enginyeria Informàtica

Escola Politècnica Superior

Año académico 2025-26

DISEÑO E IMPLEMENTACIÓN DE UNA PLATAFORMA WEB PARA LA VENTA DE PRODUCTOS DIGITALES CON GESTIÓN DE LICENCIAS Y PAGOS INTEGRADOS

Joaquin Esperon Solari

Trabajo de Fin de Grado

Escola Politècnica Superior

Universitat de les Illes Balears

Año académico 2025-26

Paraules clau del treball: plataforma web, productos digitales, gestión de licencias, Next.js, Supabase, Stripe

Tutor: Cristina Suemay Manresa Yee

Gracias a mi familia, por el apoyo a lo largo de todos estos años.

A los profesores y profesoras de la EPS que han compartido su conocimiento y han estado presentes cuando se los necesitaba. Y a todo el personal de la UIB que trabaja cada día para que miles de estudiantes puedan estudiar en un entorno cuidado y agradable.

ÍNDICE

Índice	iii
Acrónimos	vii
Resumen	ix
1 Introducción	1
1.1 Objetivos	3
1.1.1 Objetivo principal	3
1.1.2 Objetivos específicos	3
1.1.3 Objetivos personales	4
1.2 Alcance	5
1.3 Estructura del documento	5
2 Metodología	7
2.1 Metodología de desarrollo	7
2.2 Planificación y fases del proyecto	8
2.3 Herramientas de desarrollo	10
2.4 Gestión de la calidad	11
3 Marco tecnológico	13
3.1 Plataformas de venta de productos digitales	13
3.2 Arquitecturas web para aplicaciones de comercio electrónico	13
3.2.1 Modelos de renderizado	13
3.2.2 Estilos de API	13
3.2.3 Arquitectura monolítica vs. microservicios	13
3.3 Tecnologías del stack seleccionado	13
3.3.1 Next.js 15	13
3.3.2 Supabase	13
3.3.3 Stripe	14
3.3.4 Tailwind CSS y shadcn/ui	14
3.4 Seguridad en plataformas de comercio electrónico	14
3.5 Normativa aplicable	14
4 Análisis y diseño del sistema	15
4.1 Requisitos del sistema	15
4.1.1 Requisitos funcionales	15

4.1.2	Requisitos no funcionales	15
4.2	Arquitectura del sistema	15
4.3	Diseño de la base de datos	15
4.4	Diseño de la API	15
4.5	Diseño de la interfaz de usuario	15
5	Implementación	17
5.1	Entorno de desarrollo y metodología	17
5.2	Estructura del proyecto	17
5.3	Sistema de autenticación	17
5.4	Catálogo y tienda pública	17
5.5	Sistema de pagos con Stripe	17
5.6	Sistema de gestión de licencias	17
5.7	Panel de administración	17
5.8	Course viewer	18
5.9	Emails transaccionales	18
5.10	Decisiones técnicas destacadas	18
5.10.1	RLS infinite recursion (error 42P17)	18
5.10.2	Rate limiting en la API pública	18
5.10.3	Compatibilidad de Stripe API v2026	18
6	Pruebas y validación	19
6.1	Estrategia de pruebas	19
6.2	Pruebas del flujo de compra	19
6.3	Pruebas de la API pública	19
6.4	Pruebas de seguridad	19
6.5	Gestión de errores y casos límite	19
7	Resultados y discusión	21
7.1	Funcionalidades implementadas	21
7.2	Métricas de rendimiento	21
7.3	Capturas del sistema en funcionamiento	21
7.4	Comparación con soluciones existentes	21
7.5	Discusión	21
8	Conclusiones y trabajo futuro	23
8.1	Cumplimiento de objetivos	23
8.2	Conclusiones principales	23
8.3	Limitaciones del trabajo	23
8.4	Trabajo futuro	23
A	Manual de instalación y despliegue	25
A.1	Prerrequisitos	25
A.2	Configuración de variables de entorno	25
A.3	Instalación y ejecución local	25
A.4	Migraciones de base de datos	25
A.5	Despliegue en Vercel	25

B Documentación de la API pública	27
B.1 Verificar licencia	27
B.2 Activar licencia	27
B.3 Desactivar licencia	27
B.4 Ejemplo de integración en Python	27
C Esquema completo de la base de datos	29
Bibliografía	31

ACRÓNIMOS

API Application Programming Interface

CRUD Create, Read, Update and Delete

HMAC Hash-based Message Authentication Code

IDOR Insecure Direct Object Reference

IVA Impuesto sobre el Valor Añadido

OAuth Open Authorization

REST Representational State Transfer

RLS Row Level Security

SQL Structured Query Language

TFG Trabajo de Fin de Grado

URL Uniform Resource Locator

RESUMEN

[Redactar el resumen en español aquí.]

Abstract

[Write the abstract in English here.]

INTRODUCCIÓN

Los productos digitales son bienes intangibles que se crean, distribuyen y consumen íntegramente en línea, sin inventario físico, sin logística de envío y sin necesidad de almacenamiento. Una vez producidos, pueden venderse un número ilimitado de veces sin coste adicional por unidad, lo que diferencia este modelo de cualquier negocio basado en bienes físicos. La categoría engloba productos muy distintos entre sí: software y herramientas (aplicaciones, *plug-ins*, plantillas para herramientas como Notion o Google Sheets), contenido educativo (libros electrónicos, cursos en línea, guías, talleres), contenido creativo (fotografías, ilustraciones, música, vídeos) y servicios de membresía como sitios web de acceso restringido, boletines o comunidades privadas.

El mercado de productos digitales ha experimentado un crecimiento sostenido en los últimos años, impulsado por varios factores convergentes. La penetración del acceso a internet ha alcanzado cotas históricas a nivel global, lo que permite a un creador llegar a clientes en cualquier parte del mundo desde el primer día y sin infraestructura logística. Al mismo tiempo, la consolidación del trabajo remoto y la denominada *economía del creador* han llevado a un número creciente de profesionales a buscar modelos de negocio que puedan operarse íntegramente desde un ordenador, lo que casa de manera natural con la venta de productos digitales. A esto se suma una tendencia más sutil pero significativa: ante la sobreabundancia de información gratuita disponible en internet, lo que se vende con frecuencia ya no es la información en sí, sino su curación, organización y presentación. Buena parte de los cursos y guías digitales que se comercializan se sustentan en este principio.

Para el vendedor, los productos digitales presentan ventajas evidentes frente a los físicos. Carecen de coste de producción unitario: vender una copia o mil copias supone esencialmente el mismo gasto, dejando al margen la inversión en marketing. El stock es prácticamente infinito y la entrega al cliente es inmediata. La escalabilidad del negocio no exige ampliar la estructura productiva ni contratar personal adicional, sino únicamente alcanzar a más compradores. Estas características han propiciado la aparición de un ecosistema maduro de plataformas que ofrecen al creador la infraestructura necesaria para vender sus productos sin desarrollarla por su cuenta.

Entre las plataformas más representativas se encuentran Gumroad [1], Lemon Squeezy [2], Payhip [3], Sellfy y Whop. Para tiendas más generales que también admiten productos digitales destaca Shopify. Todas ellas permiten al vendedor crear un perfil, publicar productos con sus descripciones e imágenes, configurar precios y términos de uso, y procesar los pagos a través de sus pasarelas integradas. Ofrecen además paneles con métricas básicas de ventas y herramientas elementales de marketing. Su valor principal está en abstraer al vendedor de la complejidad técnica que implica montar una infraestructura de comercio electrónico, especialmente la gestión de pagos y el cumplimiento normativo.

Este modelo, sin embargo, presenta limitaciones que se hacen más visibles a medida que el negocio crece o cuando se utiliza para casos de uso menos convencionales. La primera limitación es de orden económico: las comisiones son significativas. Gumroad aplica un 10 % más una tarifa fija de 0,50 \$ por transacción para ventas a través del perfil propio del vendedor, porcentaje que asciende al 30 % cuando la venta llega a través del descubrimiento dentro de su *marketplace* [4]. Lemon Squeezy aplica un 5 % más 0,50 \$ por transacción [5]. Para volúmenes de venta elevados, este coste recurrente justifica por sí solo la inversión inicial en una solución propia.

La segunda limitación afecta al control sobre los datos del negocio. Las plataformas centralizadas exponen al vendedor únicamente la información que ellas deciden, a través de sus Application Programming Interfaces (APIs), con los filtros y formatos que ellas definen. No es posible ejecutar consultas arbitrarias sobre los propios datos, cruzarlos con fuentes externas dentro de la plataforma o acceder a metadatos internos. Más allá de la limitación técnica, existe un riesgo operativo concreto: si la plataforma cambia sus términos, suspende una cuenta o cesa su actividad, el vendedor pierde el acceso a su histórico. En la práctica se han dado casos en ambas direcciones: modificaciones de modelo de precios sin previo aviso y suspensiones temporales de cuentas que dejaron a los afectados sin acceso a sus datos durante el período de revisión.¹

La tercera limitación, y la que motivó directamente el desarrollo del presente trabajo, es la falta de soporte adecuado para productos de software con licencias activables por dispositivo. Cuando un vendedor distribuye un programa que el cliente debe ejecutar en su propio equipo (un *script* en Python, un *bot* de *trading* escrito en MQL5 (*MetaQuotes Language 5*), una aplicación de escritorio), surge la necesidad de un sistema que permita verificar desde el propio software si el usuario tiene una licencia válida y limitar el número de máquinas en las que puede activarse. La mayor parte de las plataformas analizadas no ofrecen una API pública sencilla para esta verificación. Whop, una plataforma con creciente cuota de mercado, requiere un flujo Open Authorization (OAuth) completo desde el lado del cliente, lo que complica notablemente la integración cuando el software no es una aplicación web sino un proceso ejecutándose dentro de un programa de terceros como MetaTrader 5 (plataforma de análisis y *trading* algorítmico de MetaQuotes Software). Gumroad ofrece un *endpoint* público de verificación, pero su modelo no contempla el control de activaciones por máquina, el bloqueo de claves desde un panel de administración ni un histórico auditable de eventos por licencia.

¹ Entre los casos reportados por la comunidad de vendedores digitales se encuentran la eliminación del plan gratuito de Gumroad en mayo de 2023 y diversas suspensiones de cuentas en Lemon Squeezy durante 2024, documentadas en foros y redes sociales por los propios afectados.

A todo lo anterior se suma una motivación de carácter más personal. A título personal, desarrollo y comercializo programas de *trading* algorítmico escritos en MQL5, que se ejecutan como *expert advisors* dentro de MetaTrader 5. La intención de comercializar este tipo de producto fue el detonante inicial del proyecto: ante la necesidad de un sistema de verificación de licencias, me planteé si tenía sentido pagar comisiones recurrentes a una plataforma intermediaria o desarrollar una solución propia que cubriera tanto las necesidades inmediatas como las futuras. Adicionalmente, identifiqué un segundo caso de uso dentro del entorno cercano: un proyecto familiar de venta de cursos, manuales y planos para la construcción de estructuras de madera en jardín (huertos elevados, casetas para almacenaje, fogoneros, mini-casas), susceptible de utilizar la misma plataforma para vender productos de tipo curso, libro electrónico y plantilla. Esta diversidad de casos de uso reales reforzó la decisión de plantear una plataforma con soporte heterogéneo para varios tipos de producto digital, en lugar de una solución específica para un único formato.

1.1 Objetivos

1.1.1 Objetivo principal

El objetivo principal del presente Trabajo de Fin de Grado (TFG) es el diseño y la implementación de una plataforma web para la venta de productos digitales que integre, sin depender de servicios de terceros para los aspectos críticos del negocio, la gestión del ciclo de vida completo de las licencias, una pasarela de pagos consolidada, la entrega segura de archivos y un panel de administración propio. La plataforma debe dar soporte específico a cuatro tipos de producto con requisitos distintos: software con licencias activables por dispositivo, libros electrónicos, plantillas y cursos en línea. El resultado entregado al final del trabajo es una plataforma funcional, desplegada en un entorno de producción, que pueda servir como base reutilizable para distintos negocios de venta de productos digitales mediante personalización de la marca y los contenidos.

1.1.2 Objetivos específicos

A partir del objetivo principal se han definido los siguientes objetivos específicos, formulados de manera que su consecución pueda verificarse de forma objetiva al finalizar el trabajo:

1. **OE-01.** Implementar un sistema de autenticación de usuarios con soporte para credenciales de correo electrónico y contraseña, autenticación mediante OAuth con Google y un flujo completo de recuperación de contraseña.
2. **OE-02.** Diseñar un catálogo público de productos con soporte para cuatro tipos diferenciados (*software*, *ebook*, *template* y *course*), cada uno con su propio comportamiento posterior a la compra.
3. **OE-03.** Integrar una pasarela de pagos del mercado con soporte para pagos únicos, suscripciones recurrentes con periodicidad mensual o anual y períodos de prueba con o sin recogida previa de tarjeta.

4. **OE-04.** Implementar un sistema de gestión de licencias con ciclo de vida completo (emisión, activación por dispositivo, suspensión por impago, recuperación, cancelación, expiración y revocación) y un histórico auditable de todos los eventos asociados a cada licencia.
5. **OE-05.** Desarrollar una API Representational State Transfer (REST) pública que permita a aplicaciones de terceros, sin sesión de usuario, verificar la validez de una licencia, registrar la activación en una nueva máquina y desactivar una máquina previamente registrada, con limitación de tasa para prevenir abusos.
6. **OE-06.** Garantizar la entrega segura de los archivos asociados a cada producto mediante Uniform Resource Locators (URLs) firmadas con tiempo de expiración limitado, accesibles únicamente para usuarios con una licencia activa para dicho producto.
7. **OE-07.** Implementar un visor integrado de cursos en línea con soporte para vídeo embebido (YouTube y Vimeo), contenido textual con formato Markdown, archivos adjuntos por lección y seguimiento del progreso del usuario.
8. **OE-08.** Diseñar un sistema de notificaciones por correo electrónico transaccional para los eventos clave del ciclo de vida de las licencias: confirmación de compra, fallo de pago, recuperación de pago, cancelación de suscripción y expiración inminente del periodo de prueba.
9. **OE-09.** Desarrollar un panel de administración que permita la gestión completa de productos, planes de precio, currículos de cursos, licencias, órdenes, cupones de descuento, reseñas y clientes.

1.1.3 Objetivos personales

Más allá del resultado técnico, este trabajo persigue un conjunto de objetivos personales relacionados con la consolidación de las competencias adquiridas a lo largo del grado y con la incorporación de tecnologías de uso actual en el entorno profesional.

En primer lugar, se busca aplicar los conocimientos adquiridos en *Bases de Datos I* (21716) y *Bases de Datos II* (21726) sobre un esquema relacional no trivial, en el que decisiones como el diseño de claves primarias compuestas, la elección de índices de rendimiento, las restricciones de integridad referencial o el control de acceso a nivel de fila tienen consecuencias directas y medibles sobre el comportamiento del sistema. En segundo lugar, se persigue poner en práctica lo aprendido en *Aplicaciones Distribuidas en Internet e Interfaces de Usuario* (21724) sobre un *framework* de desarrollo web actual, en el que el frontend y el backend coexisten en un mismo proyecto, con renderizado del lado del servidor, comunicación con servicios externos y consideraciones de seguridad propias del desarrollo web profesional. En tercer lugar, la asignatura *Gestión de Proyectos* (21723) ofreció el marco para abordar la planificación del desarrollo de un sistema de envergadura considerable, con múltiples fases interdependientes y la necesidad de adaptar el plan inicial a problemas técnicos no previstos.

A nivel tecnológico, se buscó deliberadamente trabajar con un *stack* representativo del estado actual del desarrollo web profesional. Esto incluye Next.js como *framework* de aplicación con renderizado del lado del servidor y API *routes* integradas, TypeScript

como lenguaje de programación para todo el proyecto, Supabase como plataforma *backend-as-a-service* basada en PostgreSQL, Stripe como pasarela de pagos y Tailwind CSS junto con *shadcn/ui* para la construcción de la interfaz de usuario. Profundizar en estas tecnologías, y especialmente en la integración con una pasarela de pagos del mercado, formaba parte de los intereses formativos del autor desde el inicio del proyecto, dada la relevancia de los pagos en línea como industria y su presencia transversal en gran parte del software comercial.

1.2 Alcance

El alcance del presente trabajo se centra en el diseño, la implementación y el despliegue de una plataforma web completa para la venta de productos digitales por parte de un único vendedor o administrador. El sistema cubre todo el ciclo desde la publicación de un producto hasta su consumo por parte del cliente final: gestión del catálogo, pasarela de pagos, generación y administración de licencias, entrega segura de archivos, visor de cursos integrado, notificaciones por correo electrónico y panel de administración. Los pagos se procesan a través del modo de pruebas de Stripe, suficiente para validar el flujo íntegro de compra, suscripciones, períodos de prueba y *webhooks* sin manejar dinero real. La plataforma se despliega sobre Vercel para el código de aplicación y Supabase para la base de datos, autenticación, almacenamiento de archivos y funciones programadas, con envío real de correos electrónicos a través de Resend.

Quedan fuera del alcance del trabajo varios aspectos que se identifican explícitamente para delimitar el ámbito de la evaluación. La plataforma está concebida para un único vendedor, por lo que no se implementa un modelo de *marketplace* con múltiples vendedores independientes. No se desarrolla aplicación móvil nativa: el acceso es exclusivamente web, si bien la interfaz se adapta a pantallas de tamaño reducido. No se aborda el cálculo y la recaudación automática de impuestos por jurisdicción geográfica (Impuesto sobre el Valor Añadido (IVA), *sales tax*), funcionalidad que Stripe ofrece como producto adicional pero cuya integración trasciende el alcance académico de este trabajo. No se han implementado pruebas unitarias ni de integración automatizadas: la validación se ha realizado de forma manual y *end-to-end*, dado que la mayor parte de la lógica del sistema reside en la interacción con servicios externos (Supabase, Stripe) cuyo comportamiento no resulta sencillo de aislar mediante *mocks* sin perder fidelidad respecto al sistema real. La interfaz de usuario está disponible únicamente en inglés; no se implementa internacionalización. Por último, el panel de administración no incluye funcionalidades de *analytics* avanzado con gráficos de evolución temporal de ingresos, un sistema de afiliados o referidos, ni soporte de chat o tickets de incidencias integrado.

1.3 Estructura del documento

La presente memoria se organiza en ocho capítulos. Tras el presente capítulo introductorio, el capítulo 2 describe la metodología de desarrollo adoptada, las fases en las que se ha estructurado el trabajo y las herramientas utilizadas durante el proceso. El capítulo 3 presenta el marco tecnológico, analiza con mayor profundidad las plataformas existentes en el mercado y justifica las decisiones técnicas adoptadas en cuanto al *stack* de desarrollo. El capítulo 4 aborda el análisis de requisitos y el diseño del sistema,

incluyendo la arquitectura general, el modelo de datos, el diseño de la API y el diseño de la interfaz de usuario. El capítulo 5 describe la implementación de las funcionalidades clave del sistema, deteniéndose en las decisiones técnicas más relevantes y en los problemas resueltos durante el desarrollo. El capítulo 6 expone la estrategia de pruebas seguida y las métricas de rendimiento obtenidas. El capítulo 7 presenta los resultados del trabajo, incluyendo la comparación entre las funcionalidades planificadas y las finalmente entregadas. Por último, el capítulo 8 recoge las conclusiones del trabajo, identifica las limitaciones detectadas y propone líneas de trabajo futuro.

METODOLOGÍA

Este capítulo describe el proceso de trabajo seguido durante el desarrollo del proyecto: el enfoque metodológico adoptado, la planificación efectiva en fases, las herramientas utilizadas como soporte al desarrollo y los mecanismos aplicados para garantizar la calidad del sistema resultante. Las tecnologías concretas que componen la plataforma se presentan en el capítulo 3; aquí se aborda el *cómo* se trabajó, no el *con qué* se construyó.

2.1 Metodología de desarrollo

El desarrollo se ha llevado a cabo siguiendo un enfoque iterativo e incremental, sin adopción de una metodología formal específica como Scrum, Kanban o XP. La decisión responde a las características concretas del proyecto: un único desarrollador, alcance acotado en el tiempo y ausencia de un equipo de trabajo. Bajo estas condiciones, el coste asociado a las ceremonias y la gestión propias de un marco formal —planificación de *sprints*, reuniones periódicas, documentación exhaustiva de tareas, métricas de equipo— no se justifica frente al beneficio que aportan en proyectos de mayor envergadura.

En su lugar, se adoptó un ciclo de trabajo sencillo organizado en torno a la entrega de funcionalidades completas. Cada iteración consistía en identificar una funcionalidad necesaria, analizar las opciones de diseño, implementarla hasta dejarla operativa y, antes de pasar a la siguiente, validarla manualmente sobre la aplicación en ejecución. Las dependencias técnicas entre funcionalidades dictaron el orden natural del desarrollo: en primer lugar la infraestructura básica, sobre ella la autenticación, después el catálogo público, a continuación los pagos y la gestión de licencias, y así sucesivamente hasta llegar a las funcionalidades más avanzadas. Este orden no se fijó formalmente desde el inicio, sino que se reveló como el único razonable a medida que cada bloque construía sobre los anteriores.

Para el seguimiento del progreso se optó por un sistema mínimo basado en un

fichero `todo.md` mantenido dentro del propio repositorio. Cada funcionalidad pendiente se registraba como una entrada de la lista; al completarse se marcaba como terminada, y las nuevas ideas que iban surgiendo durante el desarrollo se añadían al final. Esta opción presenta ventajas evidentes respecto a herramientas externas de gestión: no requiere ninguna aplicación adicional, queda versionada junto al código, resulta inmediatamente accesible desde el editor y permite editar el estado del proyecto con la misma fluidez con la que se edita cualquier fichero de texto. Como contrapartida, no conserva información temporal precisa —fechas de creación o finalización de tareas— ni permite consultar cómodamente el cronograma exacto del desarrollo. Esta limitación se asumió de forma deliberada: para un proyecto unipersonal de duración acotada, la simplicidad y la fricción mínima resultaban más valiosas que el detalle de la trazabilidad.

2.2 Planificación y fases del proyecto

El proyecto no contó con una planificación detallada desde el inicio. La idea de partida era acotada: construir una plataforma web para la venta de productos digitales con soporte propio para verificación de licencias, sobre el *stack* tecnológico que se justifica en el capítulo 3. El alcance concreto —comportamiento exacto del panel de administración, estructura del *dashboard* del usuario final, flujos específicos asociados a cada tipo de producto— fue tomando forma a medida que avanzaba el desarrollo y se identificaban necesidades reales al uso del sistema.

A posteriori, el desarrollo puede estructurarse en una fase inicial de nueve etapas consecutivas que construyeron la plataforma base, seguida de un período de refinamientos iterativos en el que se incorporaron funcionalidades adicionales y se corrigieron limitaciones detectadas durante el uso del sistema. Las nueve etapas iniciales, dictadas por las dependencias técnicas entre los distintos componentes, fueron las siguientes:

1. **Infraestructura base.** Configuración del *stack* tecnológico, definición del esquema inicial de la base de datos, sistema de tipos sobre las tablas de Supabase y separación entre los clientes de servidor y de navegador.
2. **Autenticación y layout.** Sistema de registro y acceso, integración del flujo OAuth con Google, *middleware* de protección de rutas y estructura básica de la aplicación.
3. **Administración del catálogo.** Create, Read, Update and Delete (CRUD) de productos y planes de precio desde el panel de administración, con soporte para los distintos tipos de producto.
4. **Tienda pública.** *Storefront* público, página de listado de productos y página de detalle con presentación de los planes asociados.
5. **Pagos y generación de licencias.** Integración con Stripe Checkout, implementación de los manejadores de *webhook*, generación de claves de licencia y soporte para pagos únicos y períodos de prueba iniciales.

6. **API pública de licencias.** *Endpoints* de verificación, activación y desactivación con limitación de tasa, pensados para ser consumidos desde software de terceros sin sesión de usuario.
7. **Dashboard de usuario.** Vista de licencias y órdenes, perfil de usuario y descarga de archivos asociados a las compras.
8. **Panel de administración completo.** Gestión avanzada de licencias (revocación, suspensión, recuperación), órdenes, cupones de descuento, reseñas y clientes.
9. **Escalabilidad y rendimiento.** Definición de índices sobre las tablas con mayor volumen de consulta, política de cacheado en la API pública y pruebas de carga para validar los objetivos de latencia.

Finalizadas estas etapas, el desarrollo continuó durante varias semanas en un modo de iteración incremental, en el que se incorporaron funcionalidades que no estaban previstas en la planificación inicial y se reelaboraron componentes que no se ajustaban completamente al uso esperado. Entre las funcionalidades añadidas en esta fase destacan el visor integrado de cursos en línea, la cancelación de suscripciones por parte del usuario, el manejo de pagos fallidos y la suspensión automática de licencias afectadas, el sistema de notificaciones transaccionales por correo electrónico y la función programada de avisos de expiración del período de prueba.

El carácter emergente de los requisitos se manifestó de diversas formas a lo largo del proceso. Algunas funcionalidades no estaban previstas al inicio y se incorporaron al detectarse su utilidad: el visor integrado de cursos, por ejemplo, fue una decisión tomada durante el proyecto al comprobar que su implementación sobre vídeos embebidos de YouTube y Vimeo era razonablemente sencilla y aportaba valor diferencial frente a las plataformas analizadas. Otras se replantearon a mitad del proceso. El caso más representativo es la funcionalidad que permitía a los usuarios descartar sus propias licencias gratuitas y de prueba: se implementó por completo y permaneció varios días en el sistema, pero un análisis posterior reveló dos problemas. Por un lado, introducía complejidad significativa en el control de los períodos de prueba, al permitir a un usuario reclamar repetidamente el mismo *trial* descartando el anterior. Por otro, no se corresponde con el comportamiento estándar de las plataformas analizadas, en las que una licencia gratuita simplemente permanece en la biblioteca del usuario hasta que este deja de usarla. Estas consideraciones llevaron a eliminar la funcionalidad y a simplificar el modelo de gestión de licencias.

Hubo igualmente componentes que necesitaron reescritura tras una primera implementación. El *middleware* de protección de rutas administrativas, por ejemplo, se implementó inicialmente usando el cliente estándar de Supabase, pero fallaba silenciosamente al combinar el *Edge Runtime* de Next.js con sesiones procedentes del flujo OAuth; resolverlo obligó a reescribir la verificación mediante llamadas directas a la API REST de Supabase usando la *service role key*. De forma similar, el sistema de períodos de prueba pasó por varias iteraciones hasta diferenciar correctamente los *trials* gestionados internamente de los *trials* nativos de Stripe con recogida diferida de tarjeta. Estos episodios, característicos de un proceso iterativo en el que las decisiones de diseño se validan contra el sistema en funcionamiento, se desarrollan con mayor detalle en el capítulo 5.

2. METODOLOGÍA

En cuanto a la dimensión temporal, la implementación se desarrolló de manera concentrada durante los meses de marzo y abril de 2026, aproximadamente dos meses de trabajo desde la configuración inicial del *stack* hasta la finalización de una versión completa de la plataforma. A partir de mayo, el esfuerzo se centró en la redacción de la presente memoria y en la incorporación de mejoras puntuales detectadas durante este proceso, sin abordar nuevas funcionalidades de gran calado. La convocatoria de entrega del 3 de julio de 2026 actuó como restricción temporal global, obligando a fijar un punto de corte en la implementación con anterioridad suficiente para dedicar el tiempo necesario a la elaboración de la memoria. La distribución aproximada de las etapas sobre el periodo de implementación se recoge en la figura 2.1.

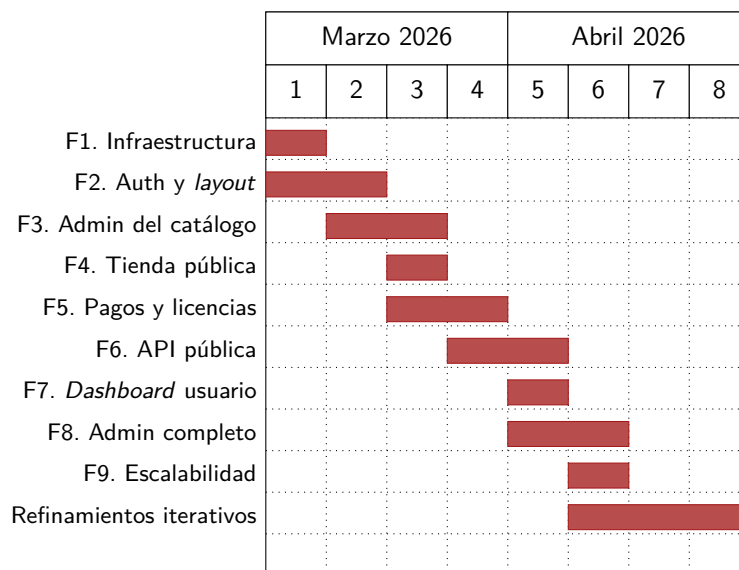


Figura 2.1: Distribución aproximada de las etapas del desarrollo sobre las ocho semanas del periodo de implementación. Las duraciones son orientativas y reflejan el orden y la solapamiento de las distintas etapas más que sus márgenes exactos.

Durante el desarrollo se mantuvo un *backlog* de funcionalidades que se decidió dejar fuera del alcance de la primera versión. La comparación entre las funcionalidades planificadas y las efectivamente entregadas se presenta en el capítulo 7.

2.3 Herramientas de desarrollo

El entorno de trabajo se construyó alrededor de un conjunto reducido de herramientas estándar en el desarrollo web moderno. Como editor se utilizó Visual Studio Code, configurado con las extensiones habituales para TypeScript, Tailwind CSS y resaltado de sintaxis Structured Query Language (SQL). El control de versiones se gestionó con Git, con el repositorio alojado en GitHub. Dado el carácter unipersonal del proyecto, no se siguió un modelo de ramas paralelas: el trabajo se realizó directamente sobre la rama principal, con *commits* frecuentes que constituyen el histórico completo del desarrollo. La motivación principal para mantener el repositorio en remoto no fue, por tanto, la

colaboración, sino la posibilidad de continuar el trabajo desde distintas máquinas y disponer de una copia de seguridad accesible.

Para el desarrollo local resultó especialmente útil la integración con servicios externos a través de herramientas dedicadas. La *CLI* oficial de Stripe (`stripe listen`) permite reenviar al servidor en ejecución local los eventos del modo de pruebas mediante un túnel autenticado, lo que resulta indispensable para depurar la lógica de los *webhooks* sin necesidad de desplegar la aplicación en cada iteración. Para la base de datos, el almacenamiento de archivos y las funciones programadas se utilizó el panel web de Supabase, que ofrece tanto un editor SQL como una interfaz para inspeccionar tablas, políticas de Row Level Security (RLS) y registros en tiempo real.

Durante el desarrollo se hizo uso de un asistente de inteligencia artificial generativa (Claude, de Anthropic) como apoyo en tareas concretas: generación de fragmentos de código, depuración de errores técnicos, consulta sobre las APIs de los servicios integrados (Supabase, Stripe, Resend) y resolución de dudas puntuales sobre el uso del *framework*. Las decisiones de diseño, la arquitectura del sistema y la selección de las funcionalidades a desarrollar correspondieron al autor del trabajo.

Como herramienta de análisis estático del código se mantuvo la configuración por defecto que proporciona Next.js, basada en ESLint con los conjuntos de reglas *core-web-vitals* (centradas en buenas prácticas de rendimiento web) y *typescript* (específicas del lenguaje). No se introdujeron reglas personalizadas adicionales: la configuración estándar resultó suficiente para detectar los errores que aparecen con mayor frecuencia durante el desarrollo.

2.4 Gestión de la calidad

En ausencia de un proceso formal de pruebas automatizadas, la calidad del sistema se sostuvo sobre varios mecanismos complementarios aplicados de manera sistemática durante el desarrollo.

El primero, y posiblemente el de mayor impacto, es el uso de TypeScript en todo el código del proyecto, junto con la definición manual de los tipos correspondientes a las tablas de Supabase. Esta práctica permite detectar en tiempo de compilación una clase importante de errores que de otro modo solo se manifestarían en ejecución: nombres de columnas incorrectos, valores nulos no contemplados, formatos de respuesta inconsistentes entre el cliente y el servidor. De manera complementaria, la validación de los datos de entrada se aborda mediante esquemas Zod en los principales formularios del cliente (integrados con *react-hook-form*) y en las rutas de API que reciben datos sensibles del usuario, garantizando que los datos procesados respondan al formato esperado antes de llegar a la lógica de negocio.

Las pruebas funcionales se realizaron de forma manual y *end-to-end* sobre el sistema en ejecución local. Al finalizar la implementación de cada funcionalidad, se ejecutaba la aplicación con `npm run dev` y se validaba el flujo completo desde la interfaz, prestando especial atención a la consistencia entre el rol de administrador y el rol de usuario final. Resulta habitual, por ejemplo, verificar que un cambio aplicado desde el panel de administración se refleja correctamente en el *dashboard* del usuario, o que un nuevo evento generado por un *webhook* de Stripe se traduce en el comportamiento esperado en la base de datos y en la interfaz. Esta validación se complementaba con

pruebas dirigidas a casos extremos: valores límite, datos inesperados en formularios o intentos de acceso a recursos pertenecientes a otro usuario. No se implementaron pruebas automatizadas, unitarias o de integración: una limitación que se reconoce explícitamente y que se discute en el capítulo 8.

El desarrollo se llevó a cabo sin un entorno de *staging* intermedio. Toda la implementación se ejecutaba localmente, con Stripe operando en su modo de pruebas y con un proyecto de Supabase dedicado al desarrollo. Esta decisión, propia de un proyecto unipersonal de duración acotada, resulta razonable mientras el sistema se valide directamente contra los servicios externos en sus modos de prueba, que reproducen con fidelidad el comportamiento de producción. El despliegue en Vercel y la transición de Stripe al modo real quedan reservados para la entrega final del trabajo.

Por lo que respecta a la seguridad, la estrategia adoptada consistió en delegar los componentes más críticos a servicios especializados y concentrar el esfuerzo propio en las capas de seguridad que corresponden al código de aplicación. El almacenamiento y el *hashing* de las contraseñas se delegan en Supabase Auth, el procesamiento de los datos de tarjeta a Stripe —cumpliendo el estándar PCI DSS— y la terminación TLS al proveedor de despliegue. Sobre estas bases, el código propio implementa los controles que afectan a la lógica de la aplicación: filtrado anti-Insecure Direct Object Reference (IDOR) de todas las rutas que devuelven recursos de usuario, verificación Hash-based Message Authentication Code (HMAC) de los *webhooks* entrantes, URLs firmadas con tiempo de expiración para la entrega de archivos, políticas de RLS a nivel de base de datos como segunda barrera frente a fallos del código de aplicación, y separación estricta entre los clientes de Supabase del lado del servidor —con acceso completo— y del lado del navegador —sujetos a las políticas de RLS—. El detalle concreto de estas medidas se desarrolla en el capítulo 4.

MARCO TECNOLÓGICO

3.1 Plataformas de venta de productos digitales

[Redactar análisis de plataformas existentes aquí.]

3.2 Arquitecturas web para aplicaciones de comercio electrónico

3.2.1 Modelos de renderizado

[Redactar comparativa de modelos de renderizado aquí.]

3.2.2 Estilos de API

[Redactar justificación de REST aquí.]

3.2.3 Arquitectura monolítica vs. microservicios

[Redactar justificación de arquitectura aquí.]

3.3 Tecnologías del stack seleccionado

3.3.1 Next.js 15

[Redactar justificación de Next.js aquí.]

3.3.2 Supabase

[Redactar justificación de Supabase aquí.]

3.3.3 Stripe

[Redactar justificación de Stripe aquí.]

3.3.4 Tailwind CSS y shadcn/ui

[Redactar justificación de Tailwind + shadcn/ui aquí.]

3.4 Seguridad en plataformas de comercio electrónico

[Redactar análisis de seguridad aquí.]

3.5 Normativa aplicable

[Redactar normativa aplicable aquí.]

ANÁLISIS Y DISEÑO DEL SISTEMA

4.1 Requisitos del sistema

4.1.1 Requisitos funcionales

[Insertar tabla de requisitos funcionales aquí.]

4.1.2 Requisitos no funcionales

[Insertar tabla de requisitos no funcionales aquí.]

4.2 Arquitectura del sistema

[Insertar diagrama y descripción de arquitectura aquí.]

4.3 Diseño de la base de datos

[Insertar diagrama ER y descripción de tablas aquí.]

4.4 Diseño de la API

[Insertar tabla de endpoints y descripción aquí.]

4.5 Diseño de la interfaz de usuario

[Insertar capturas de pantalla / wireframes y descripción de UX aquí.]

IMPLEMENTACIÓN

5.1 Entorno de desarrollo y metodología

[Redactar descripción del entorno de desarrollo aquí.]

5.2 Estructura del proyecto

[Redactar descripción de la estructura del proyecto aquí.]

5.3 Sistema de autenticación

[Redactar descripción del sistema de autenticación aquí.]

5.4 Catálogo y tienda pública

[Redactar descripción de la tienda pública aquí.]

5.5 Sistema de pagos con Stripe

[Redactar descripción del sistema de pagos aquí.]

5.6 Sistema de gestión de licencias

[Redactar descripción del sistema de licencias aquí.]

5.7 Panel de administración

[Redactar descripción del panel de administración aquí.]

5.8 Course viewer

[Redactar descripción del course viewer aquí.]

5.9 Emails transaccionales

[Redactar descripción del sistema de emails aquí.]

5.10 Decisiones técnicas destacadas

5.10.1 RLS infinite recursion (error 42P17)

[Redactar descripción del problema RLS aquí.]

5.10.2 Rate limiting en la API pública

[Redactar descripción del rate limiting aquí.]

5.10.3 Compatibilidad de Stripe API v2026

[Redactar descripción del problema de compatibilidad Stripe aquí.]

PRUEBAS Y VALIDACIÓN

6.1 Estrategia de pruebas

[Redactar descripción de la estrategia de pruebas aquí.]

6.2 Pruebas del flujo de compra

[Redactar descripción de las pruebas de compra aquí.]

6.3 Pruebas de la API pública

[Insertar resultados de la prueba de carga aquí.]

6.4 Pruebas de seguridad

[Redactar descripción de las pruebas de seguridad aquí.]

6.5 Gestión de errores y casos límite

[Redactar descripción de la gestión de errores aquí.]

RESULTADOS Y DISCUSIÓN

7.1 Funcionalidades implementadas

[Insertar tabla de funcionalidades implementadas aquí.]

7.2 Métricas de rendimiento

[Insertar métricas de rendimiento aquí.]

7.3 Capturas del sistema en funcionamiento

[Insertar capturas de pantalla aquí.]

7.4 Comparación con soluciones existentes

[Insertar tabla comparativa y análisis aquí.]

7.5 Discusión

[Redactar discusión aquí.]

CONCLUSIONES Y TRABAJO FUTURO

8.1 Cumplimiento de objetivos

[Redactar evaluación del cumplimiento de objetivos aquí.]

8.2 Conclusiones principales

[Redactar conclusiones principales aquí.]

8.3 Limitaciones del trabajo

[Redactar descripción de las limitaciones aquí.]

8.4 Trabajo futuro

[Redactar propuestas de trabajo futuro aquí.]



MANUAL DE INSTALACIÓN Y DESPLIEGUE

A.1 Prerrequisitos

[Redactar lista de prerrequisitos aquí.]

A.2 Configuración de variables de entorno

[Redactar descripción de variables de entorno aquí.]

A.3 Instalación y ejecución local

[Redactar instrucciones de instalación local aquí.]

A.4 Migraciones de base de datos

[Redactar instrucciones de migración aquí.]

A.5 Despliegue en Vercel

[Redactar instrucciones de despliegue aquí.]



DOCUMENTACIÓN DE LA API PÚBLICA

B.1 Verificar licencia

[Redactar documentación completa del endpoint verify aquí.]

B.2 Activar licencia

[Redactar documentación del endpoint activate aquí.]

B.3 Desactivar licencia

[Redactar documentación del endpoint deactivate aquí.]

B.4 Ejemplo de integración en Python

[Insertar ejemplo de integración aquí.]



ESQUEMA COMPLETO DE LA BASE DE DATOS

[Insertar esquema completo o diagrama ER de alta resolución aquí.]

BIBLIOGRAFÍA

- [1] Gumroad Inc., “Gumroad — sell what you know,” <https://gumroad.com>, 2024, consultado en 2025. 1
- [2] Lemon Squeezy LLC, “Lemon squeezy — payments, subscriptions and digital products,” <https://www.lemonsqueezy.com>, 2024, consultado en 2025. 1
- [3] Payhip Ltd., “Payhip — sell digital downloads & online courses,” <https://payhip.com>, 2024, consultado en 2025. 1
- [4] Gumroad Inc., “Gumroad pricing,” <https://gumroad.com/pricing>, 2025, consultado en 2025. 1
- [5] Lemon Squeezy LLC, “Lemon squeezy pricing,” <https://www.lemonsqueezy.com/pricing>, 2025, consultado en 2025. 1